This listing combines the four clauses using a `must` query, matching LG TVs but not silver ones (`must_not` clause). If a product has an A++ energy rating or is a fridge freezer (`should` clause), that's even better. Finally, we filter the products by price (`filter` clause).

We can enhance the query with further clauses and leaf queries as more complex requirements are added to the list. There is no restriction on how many queries a clause can contain; it is entirely up to our discretion. But how do we know which leaf queries in the various clauses matched to get results? Were any of the queries omitted based on our requirements? Knowing the exact leaf query that obtained the final outcome (aka results) is a great way to identify queries that were executed and produced results. We can do this by naming every query.

11.3.9 Named queries

We may build dozens of queries for a complex query; however, we have no clue how many are used in a match to get the final results. We can name our queries so that Elasticsearch outputs the results along with the names of the queries used during the query match. Let's look at an example. The next listing shows a complex query with all the clauses and a few leaf queries.

Listing 11.15 Complex queries given individual names

```
GET products/_search
{
  "_source": ["product", "brand"],
  "query": {
    "bool": {
      "must": [
        {
          "match": {
            "brand": {
              "query": "LG",
              "_name": "must_match_brand_query"
            }
          }
        }
      ],
      "must_not": [
        {
          "match": {
            "colour.keyword": {
              "query":"black",
              "_name":"must_not_colour_query"
            }
          }
        }
      ],
      "should": [
        {
          "term": {
```

Query name to match the brand in a must clause

Query name to not match a particular color